S3 — Passo a passo para rodar no servidor

Squad VER-3 · Componente S3 — Alertas e Notificações · Broker do projeto (Guia do Kafka da TRA-1)
05/11/2026 · Código: main no GitHub, commit dd99c5b

Resumo: o S3 está pronto para o servidor. A imagem Docker e o script de instalação foram testados (o script, num Ubuntu limpo, sem Docker), e o GitHub passa a rodar os testes sozinho a cada push. Endereço do broker: **34.66.235.91:9093**, sem autenticação; nomes dos tópicos aprovados pela TRA-1 (os da especificação). **O que falta é ter o servidor:** qual máquina e como entrar nela. Com isso, a instalação é um comando e leva uns 3 minutos de máquina ligada.

Situação

#	Etapa	Situação	Detalhe
1	Código pronto e testado	☑ FEITO	Imagem Docker testada contra o Kafka local em 04/11; 385 testes passando
2	Script de instalação testado	☑ FEITO	infra/servidor/instalar.sh rodado num Ubuntu 24.04 limpo em 04/11: instalou o Docker, ligou o S3 e processou um evento
3	Verificação mínima no broker	☑ FEITO	tools/verificar_broker.py, dentro da imagem; passou contra o Kafka local em 4 s
4	Integração contínua (GitHub Actions)	☑ FEITO	Testes, cobertura e imagem a cada push; a primeira execução no GitHub passou (04/11, commit ea1ae2d)
5	Juntar na main	☐ FALTA	A main (dd99c5b) já tem script, verificação e CI; falta a criação de tópicos de 05/11
6	Descobrir o servidor e como entrar nele	☐ FALTA	Com quem pediu o S3 no servidor
7	Confirmar dois pontos com a TRA-1	☑ FEITO	05/11: livres para criar tópicos (e o broker cria sozinho); o S3 pode ficar ligado
8	Entrar no servidor e rodar o script	☐ FALTA	No servidor: 3 comandos, uns 3 minutos
9	Conferir e rodar a verificação no broker	☐ FALTA	No servidor, logo depois do script

1. Código pronto e testado ☑ FEITO

Na máquina de desenvolvimento

A imagem foi construída e rodada contra o Kafka local, sem tocar no broker oficial. Resultado:

Verificação	Resultado
Evento do S2 vira alerta, gravado e publicado	ok
Banco no volume sobrevive ao reinício do contêiner	ok, sem reprocessar nada
`docker stop` (SIGTERM)	sai em ~1 s, fechando o consumidor e o banco
Ao religar, processa um evento novo	em ~5 s
Recursos parado	18 MB de memória, 0,04% de CPU; imagem de 200 MB
Usuário dentro do contêiner	s3 (uid 1000), não root
Suíte de testes	385 testes passando, nenhum usa o broker

2. Script de instalação testado ☑ FEITO

infra/servidor/instalar.sh

O script faz sozinho o que antes eram três etapas à mão. Teste: um contêiner Ubuntu 24.04 limpo fez o papel do servidor, e um Kafka separado fez o papel do broker oficial.

O script	No teste
Instala o Docker, se faltar	instalou (Docker 29.9 e Compose 5.7)
Guarda o broker em `infra/servidor/.env` (fora do git)	ok; depois de down e up, religou sem export
Constrói a imagem e liga o `ods-s3`	ok, contêiner Up
Cria no broker os tópicos do S3 que faltam, antes de ligar o S3	Kafka sem tópicos: criou os 9; na segunda execução, "já existe" nos 9
Mostra o log e os comandos do dia a dia	ok
S3 instalado processa um evento	evento publicado virou alerta (alerta_criado, CREATED)

3. Verificação mínima no broker ☑ FEITO

tools/verificar_broker.py (vai dentro da imagem)

Um comando, para rodar uma vez no servidor, que prova o S3 no broker real gastando o mínimo. Faz o papel do A5 (aplicação do VER-3), sem se passar pelo S2: confere os 9 tópicos, cria um alerta de teste por comando ("[TESTE S3]"), espera o ods.s3.alert, dá ACKNOWLEDGE e CLOSE (o alerta de teste não fica aberto nas aplicações) e consulta o histórico. São 4 comandos e 1 consulta; os consumidores começam do fim dos tópicos.

No teste contra o Kafka local (04/11)	Resultado
Com o S3 ligado	tudo ok em 4 s; 4 mensagens enviadas, 7 recebidas
Endereço fora de localhost sem --broker-oficial	recusa antes de conectar
Com o S3 desligado	falha em 15 s: "não chegou a resposta... O S3 está ligado?"

4. Integração contínua (GitHub Actions) ☑ FEITO

.github/workflows/ci.yml

A cada push e pull request, o GitHub roda a suíte com cobertura e constrói a imagem do servidor. Roda nos servidores do GitHub: **não gasta os créditos do professor** nem usa o broker. Os mesmos passos foram conferidos num contêiner Linux com Python 3.12: 385 testes passando, 98% de cobertura, imagem construída e S3 abrindo dentro dela. **A primeira execução no GitHub passou** (04/11, commit ea1ae2d); o resultado de cada push aparece na aba **Actions** do repositório.

5. Juntar na main ☐ FALTA

Na sua máquina (a main dd99c5b já tem o script, a verificação e o CI)

```
git add -A
git commit -m "Script de instalação cria os tópicos do S3 antes de ligar (TRA-1, 05/11)"
git push
```

Depois, juntar na main (pull request), como da outra vez. O servidor baixa o código da main.

6. Descobrir o servidor e como entrar nele ☐ FALTA

Com quem pediu o S3 no servidor (professor, TRA-1 ou P3/SIS-1)

Pergunta	Por quê
Qual máquina? (endereço IP ou nome; Google Cloud ou outra)	É para onde você vai entrar
Como entrar? (usuário e senha, ou chave SSH)	Sem acesso não há como instalar nada
Qual sistema operacional?	O script é para Ubuntu ou Debian
A máquina tem saída para a internet e para o broker?	Para baixar o código e alcançar 34.66.235.91:9093; não precisa abrir porta de entrada

7. Confirmar dois pontos com a TRA-1 ☑ FEITO

Respondido pela TRA-1 em 05/11

Pergunta	Resposta da TRA-1	O que muda
Os tópicos do S3 existem no broker?	Os squads estão livres para criar, e o broker cria sozinho na primeira mensagem	O script cria os que faltam antes de ligar o S3: um tópico criado depois que o S3 já escuta pode levar até 5 min para ser percebido
Pode deixar o S3 ligado o tempo todo?	Pode	O S3 fica ligado no servidor

8. Entrar no servidor e rodar o script ☐ FALTA

No seu terminal, depois dentro do servidor

```
ssh usuario@ENDERECO_DO_SERVIDOR
sudo apt-get install -y git                    # só se o git faltar
git clone https://github.com/ods-26-2/s3.git && cd s3
bash infra/servidor/instalar.sh 34.66.235.91:9093
```

O script usa sudo para instalar o Docker, se precisar. No fim, ele mostra os tópicos criados ou já existentes (**ok: 9 tópicos do S3 conferidos** é o esperado) e as últimas linhas do log. Para fazer tudo à mão, os comandos estão em `docs/IMPLANTACAO_SERVIDOR.md`.

9. Conferir e rodar a verificação no broker ☐ FALTA

Dentro do servidor, na pasta s3

```
docker compose -f infra/servidor/docker-compose.yml ps          # ods-s3 deve estar "Up"
docker compose -f infra/servidor/docker-compose.yml logs -f s3  # Ctrl+C sai do log; o S3 continua ligado
```

Logo depois, a verificação (etapa 3). O esperado é terminar com **Tudo certo**; tire a captura: é a evidência do S3.2 no broker real.

```
docker exec ods-s3 python tools/verificar_broker.py --broker-oficial
```

Depois, quando o S2 ou o A5 mandarem algo, o log mostra `entrada ... resultado=alerta_criado` e `saida ... state=CREATED`.

No dia a dia

Para	Comando, dentro do servidor (na pasta s3)
Ver o log	`docker compose -f infra/servidor/docker-compose.yml logs -f s3`
Parar (o banco fica guardado)	`docker compose -f infra/servidor/docker-compose.yml down`
Ligar de novo (lê o broker do .env)	`docker compose -f infra/servidor/docker-compose.yml up -d`
Atualizar o código	`bash infra/servidor/instalar.sh 34.66.235.91:9093 --atualizar`
Contar os alertas no banco	`docker exec ods-s3 python -c "import sqlite3; print(sqlite3.connect('/dados/s3.sqlite3').execute('select count(*) from alertas').fetchone())"`

Para economizar os créditos

O quê	Por quê
Usar a menor máquina possível (1 a 2 GB de RAM, como uma `e2-small` no Google Cloud)	O S3 usa 18 MB de memória e quase nada de CPU parado
Desligar a máquina (não apagar) quando ninguém estiver testando	O banco fica no disco e o S3 volta sozinho quando a máquina liga (restart: unless-stopped)
Rodar o script de uma vez, sem testar comando por comando no servidor	A instalação foi ensaiada num Ubuntu limpo: é a máquina ligada pelo menor tempo
Testes e CI no GitHub, não no servidor	O CI roda na cota gratuita do GitHub

Regras importantes

Um S3 só. Com o S3 no servidor, não ligue outro S3 contra o mesmo broker (nem na sua máquina): dois S3 respondem aos mesmos comandos, cada um com o seu banco. O `kit_aplicacoes.py` e o `teste_kafka_local.py` recusam qualquer broker que não seja localhost, de propósito.

O broker é pago e compartilhado pelos 15 squads. O S3 espera 1 s entre leituras, sem martelar o broker. A TRA-1 liberou deixar o S3 ligado (05/11).

Problemas comuns

O que aparece	Causa	O que fazer
uso: bash infra/servidor/instalar.sh <broker host:porta>	Faltou o endereço do broker	`bash infra/servidor/instalar.sh 34.66.235.91:9093`
defina S3_KAFKA_BOOTSTRAP	Compose rodado sem o .env (antes do script)	Rodar o script uma vez; ele cria o .env
permission denied ... docker.sock	O usuário ainda não está no grupo docker	Saia e entre de novo pelo ssh, ou use sudo na frente do comando
AVISO: o broker não respondeu	Rede do servidor sem saída para o broker	Falar com a TRA-1; testar a rede com `nc -vz 34.66.235.91 9093`
S3 ligado, mas nada chega no log	Ninguém publicando ainda	Normal até o S2 e o A5 publicarem
Depois de um reinício, demora até ~45 s para processar	O S3 anterior foi morto sem fechar (queda de energia, kill)	Esperar: o Kafka libera o lugar sozinho. Com docker stop ou down isso não acontece